마이크로프로세서

Chapter 6. Timer

Human-Robot Interaction Laboratory

KYUNG HEE UNIVERSITY

>> Timer

Advanced-control timers (TIM1 and TIM8)

Low-density devices are STM32F101xx, STM32F102xx and STM32F103xx microcontrollers where the Flash memory density ranges between 16 and 32 Kbytes.

Medium-density devices are STM32F101xx, STM32F102xx and STM32F103xx microcontrollers where the Flash memory density ranges between 64 and 128 Kbytes.

High-density devices are STM32F101xx and STM32F103xx microcontrollers where the Flash memory density ranges between 256 and 512 Kbytes.

XL-density devices are STM32F101xx and STM32F103xx microcontrollers where the Flash memory density ranges between 768 Kbytes and 1 Mbyte.

Connectivity line devices are STM32F105xx and STM32F107xx microcontrollers.

Low- and medium-density STM32F103xx devices, and the STM32F105xx/STM32F107xx connectivity line devices, contain one advanced-control timer (TIM1) whereas high-density and XL-density STM32F103xx devices feature two advance-control timers (TIM1 and TIM8).

General-purpose timers (TIM2 to TIM5)

Low-density devices are STM32F101xx, STM32F102xx and STM32F103xx microcontrollers where the Flash memory density ranges between 16 and 32 Kbytes.

Medium-density devices are STM32F101xx, STM32F102xx and STM32F103xx microcontrollers where the Flash memory density ranges between 64 and 128 Kbytes.

High-density devices *are STM32F101xx and STM32F103xx microcontrollers where the Flash memory density ranges between 256 and 512 Kbytes.*

XL-density devices *are STM32F101xx and STM32F103xx microcontrollers where the Flash memory density ranges between 768 Kbytes and 1 Mbyte.*

Connectivity line devices are STM32F105xx and STM32F107xx microcontrollers.

This section applies to the whole STM32F10xxx family, unless otherwise specified.

Timer

>> Timer

- STM32F103RB는 Medium-density device로 Advanced-control Timer TIM1과 General-purpose Timer TIM2, 3, 4를 지원함

- TIM1, 2, 3, 4는 16비트 카운터를 통해 동작
  - Up-counting, Down-counting, Center-aligned(Up/Down Counting)을 지원함
  - 16비트 Prescaler를 통해 카운터 클럭을 나누어 활용 가능

- 4개의 16비트 Capture/Compare 채널
  - Input Capture, Output Compare
  - PWM Generation
  - One-pulse mode output

- 채널 동작에 따른 인터럽트/DMA 요청

Timer

>> Timer

Figure 100. General-purpose timer block diagram

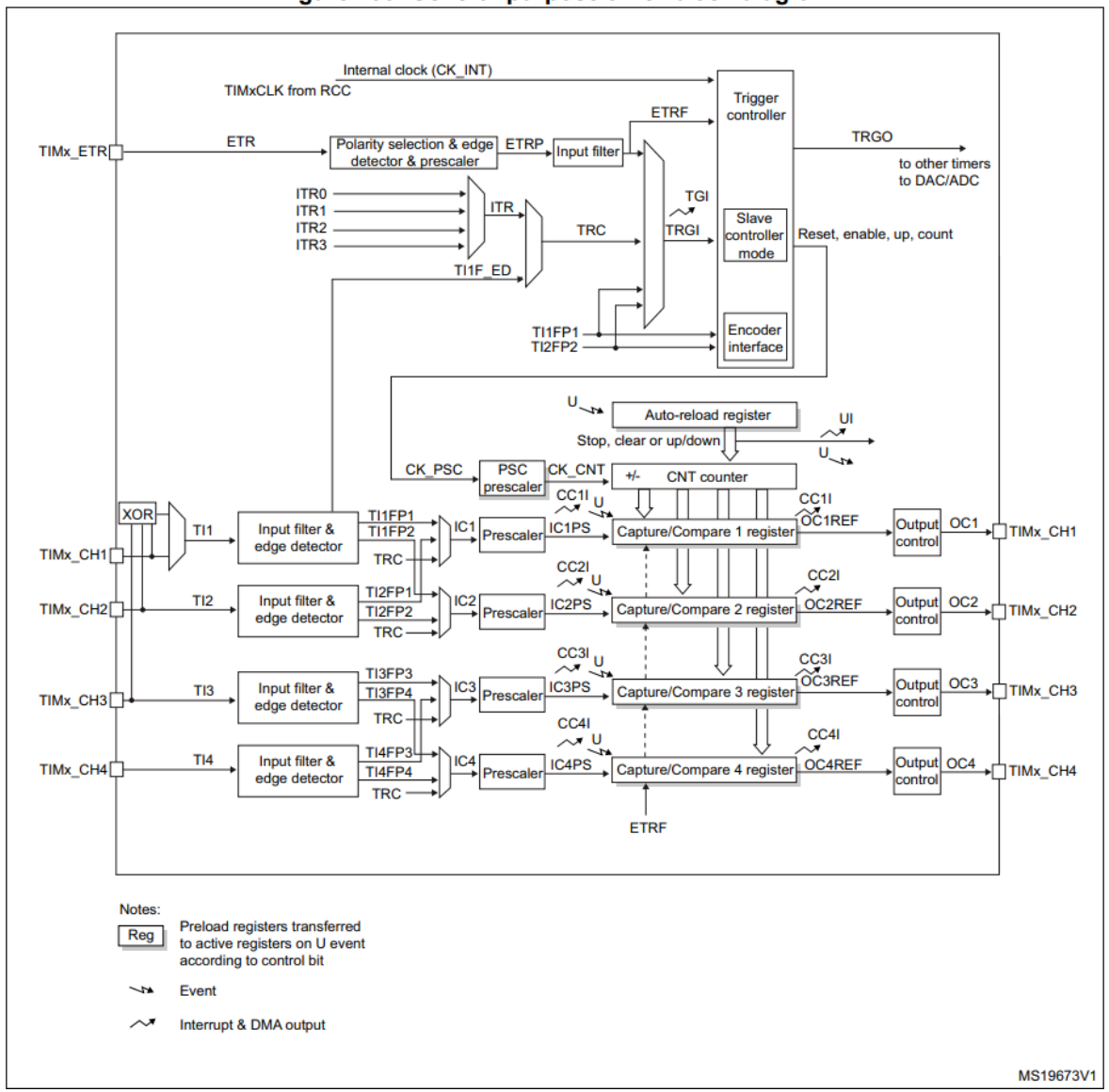
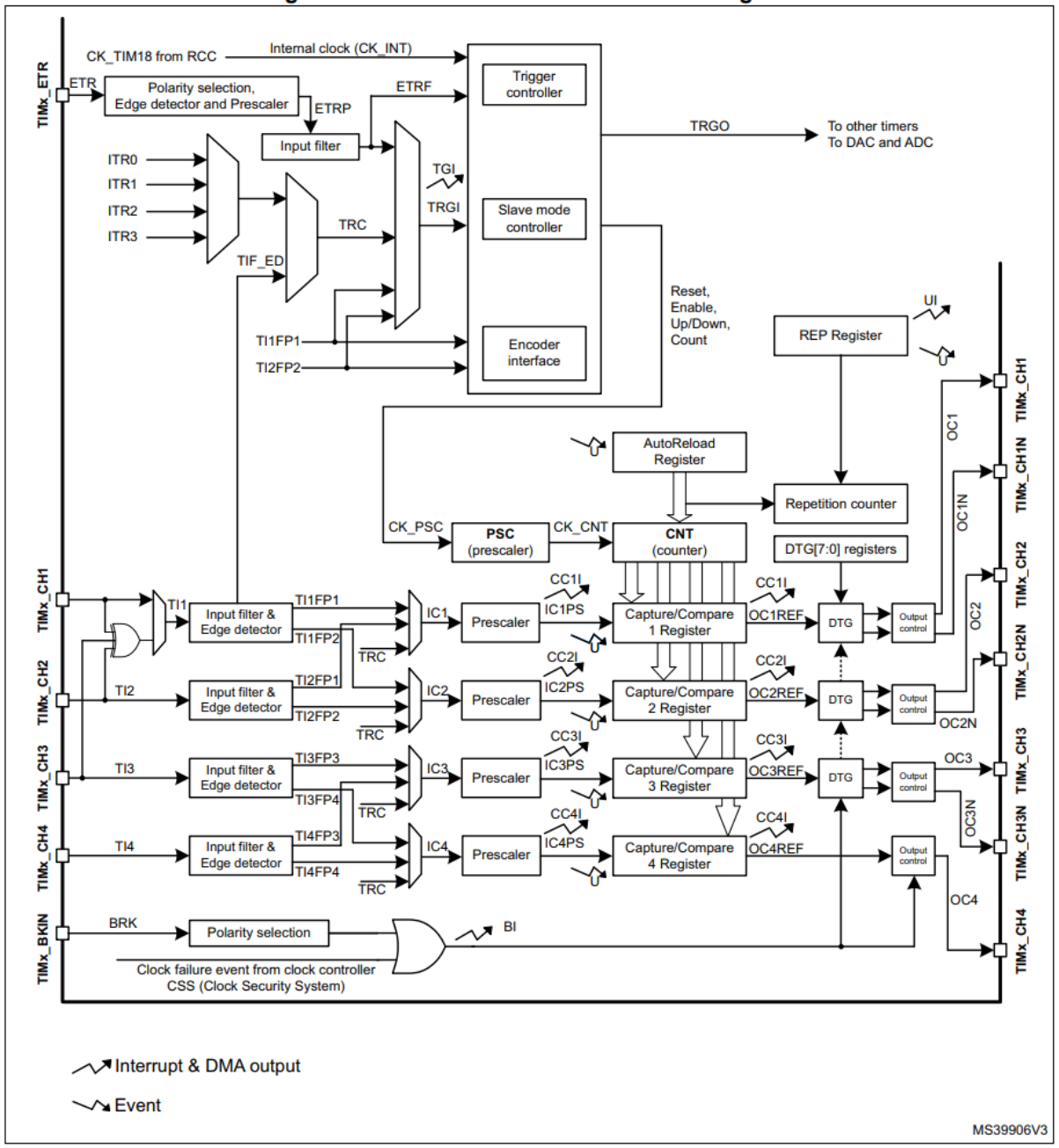


Figure 52. Advanced-control timer block diagram



Timer

>> Timer Prescaler

- Prescaler는 counter clock 주파수를 분할
- 16비트 TIMx_PSC 레지스터를 통해 1 ~ 65536 값으로 주파수를 나눌 수 있음

Figure 101. Counter timing diagram with prescaler division change from 1 to 2

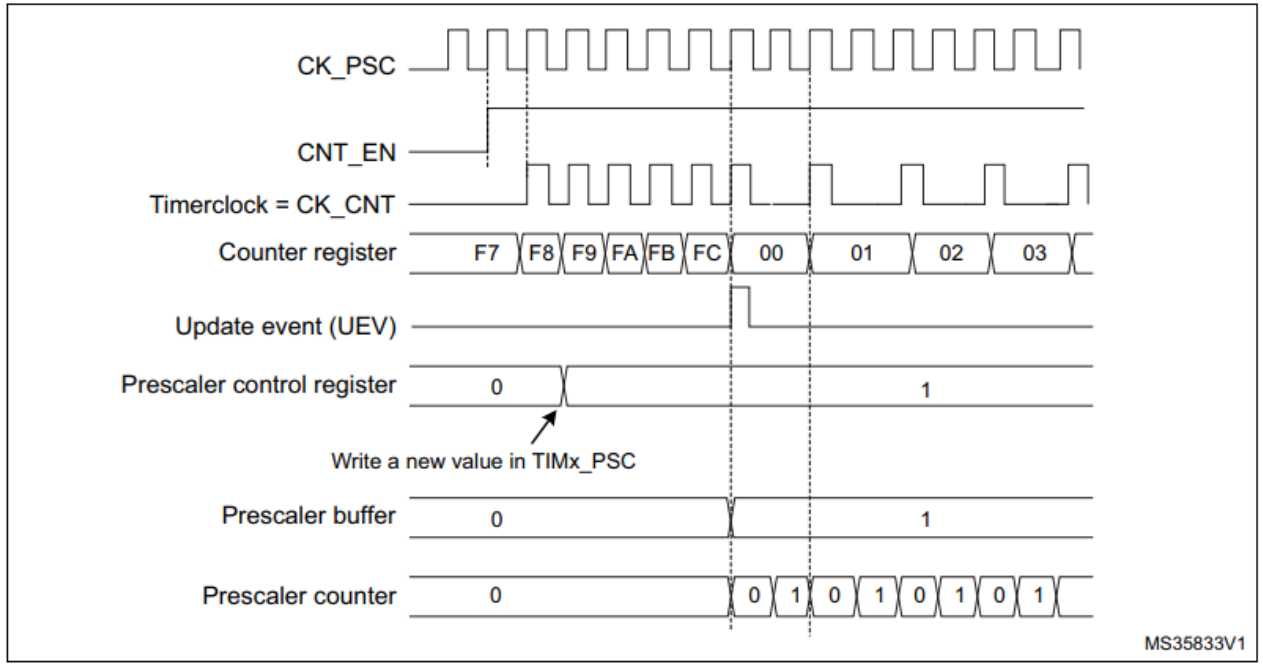
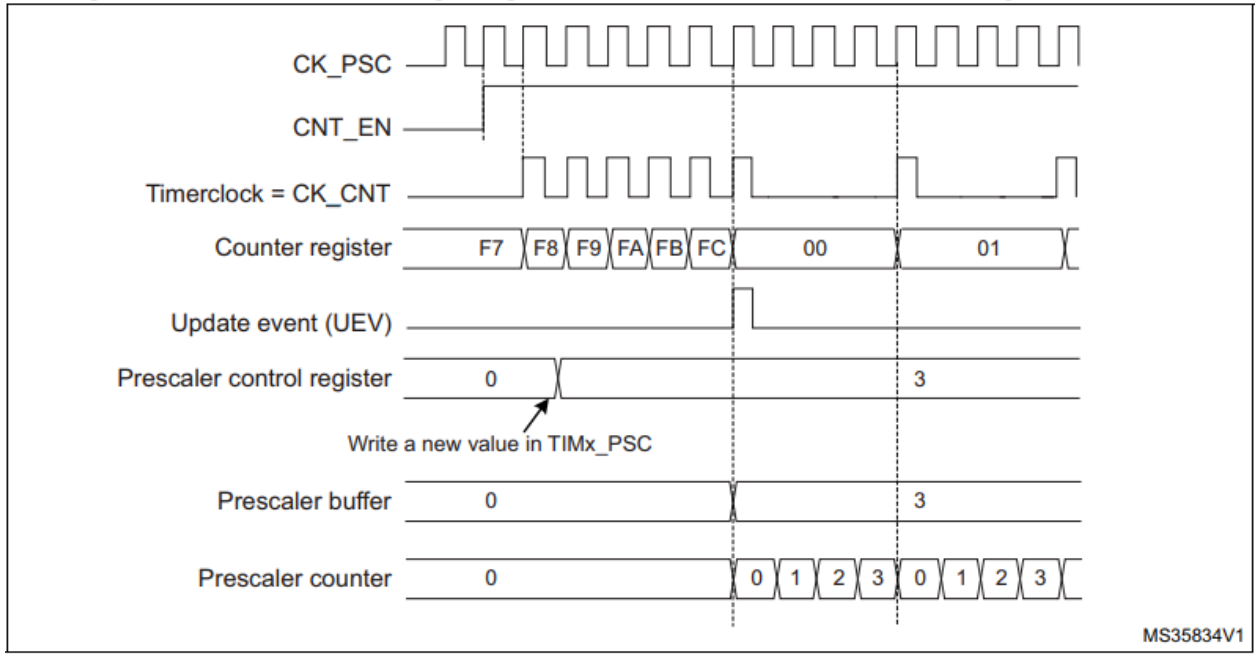


Figure 102. Counter timing diagram with prescaler division change from 1 to 4



Timer

>> Timer 동작

- **Up Counting: CNT가 0부터 TIMx_ARR(auto-reload값)까지 증가, ARR값에 도달하면 오버플로우, 업데이트 이벤트(UEV), 업데이트 플래그(UIF)가 발생**

Figure 103. Counter timing diagram, internal clock divided by 1

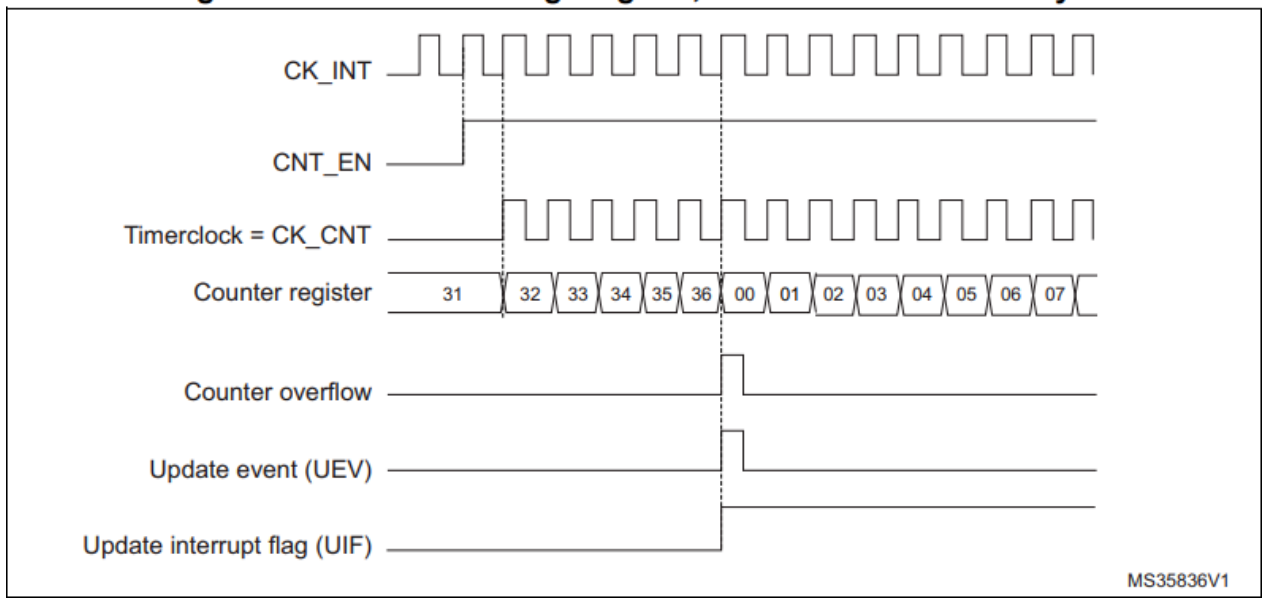
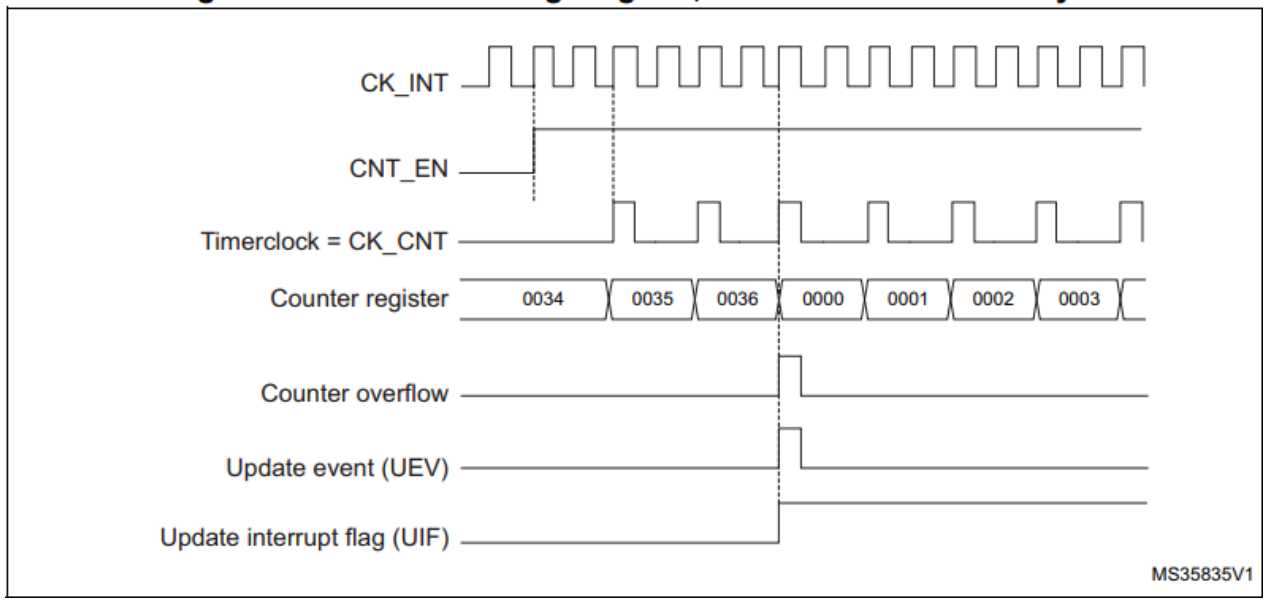


Figure 104. Counter timing diagram, internal clock divided by 2



The following figures show some examples of the counter behavior for different clock frequencies when TIMx_ARR=0x36.

>> Timer 동작

- **Down Counting: CNT값이 ARR부터 0까지 감소, 0에 도달하면 언더플로우, UEV, UI가 발생**

Figure 109. Counter timing diagram, internal clock divided by 1

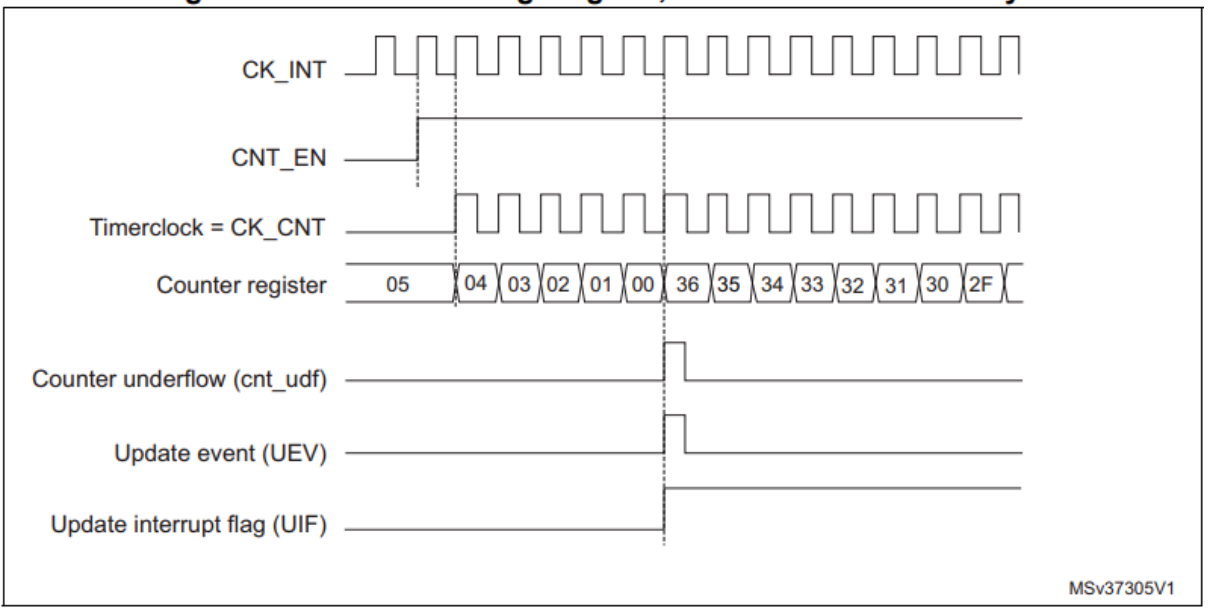
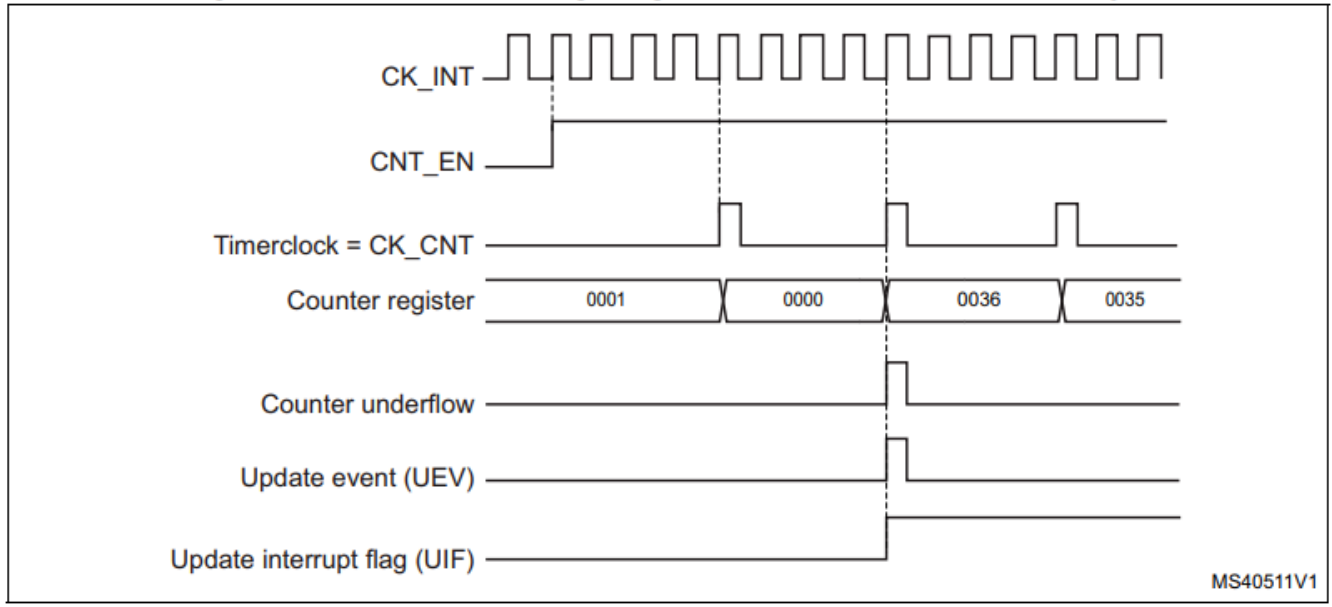


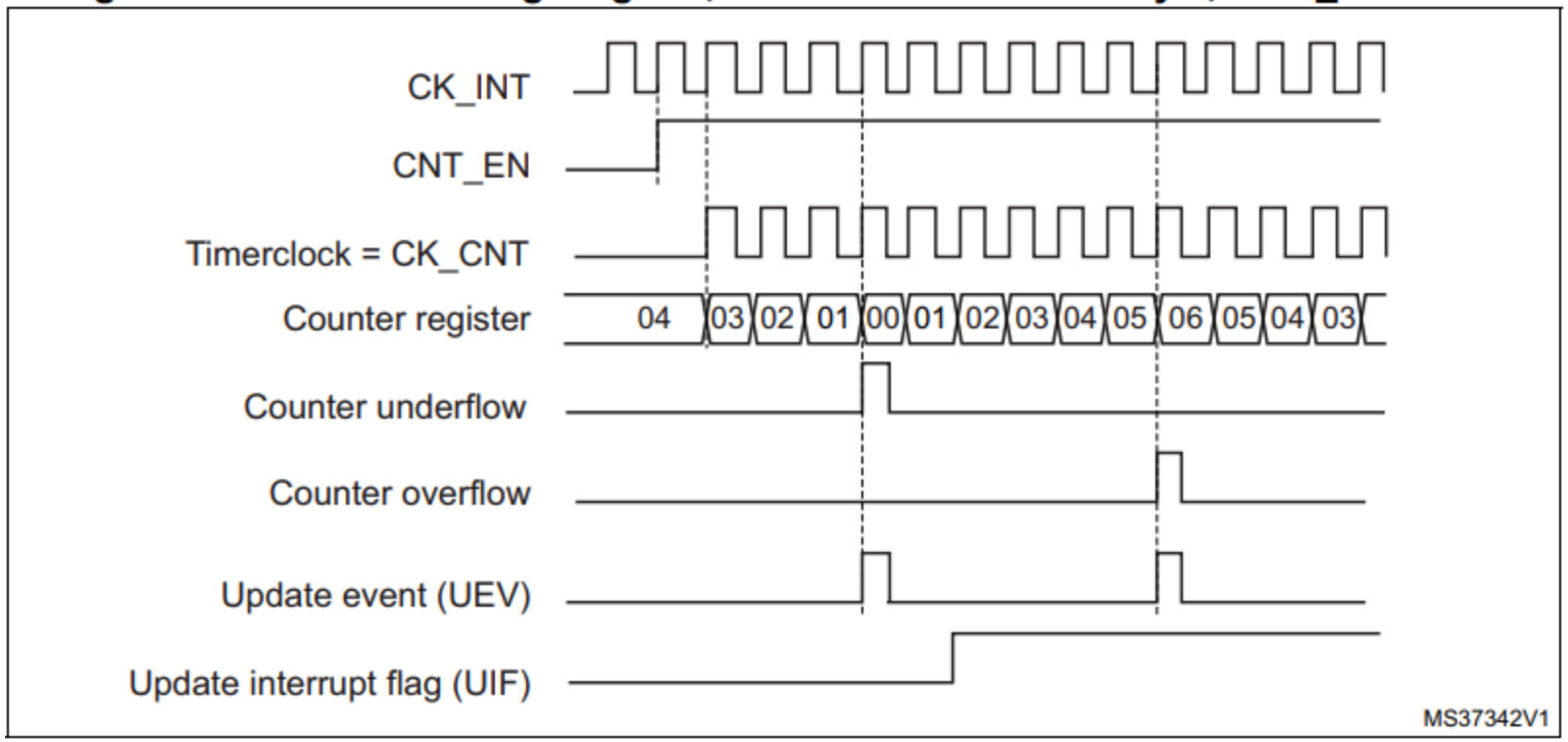
Figure 111. Counter timing diagram, internal clock divided by 4



>> Timer 동작

- Center-aligned: CNT가 0부터 ARR까지 증가한 후 ARR에서 다시 0까지 감소, 오버플로우, 언더플로우, UEV, UI 모두 발생

Figure 114. Counter timing diagram, internal clock divided by 1, TIMx_ARR=0x6



>> Timer 동작

- **Auto-reload preload enable bit(ARPE): ARR값의 반영을 결정**

Figure 107. Counter timing diagram, Update event when ARPE=0 (TIMx_ARR not preloaded)

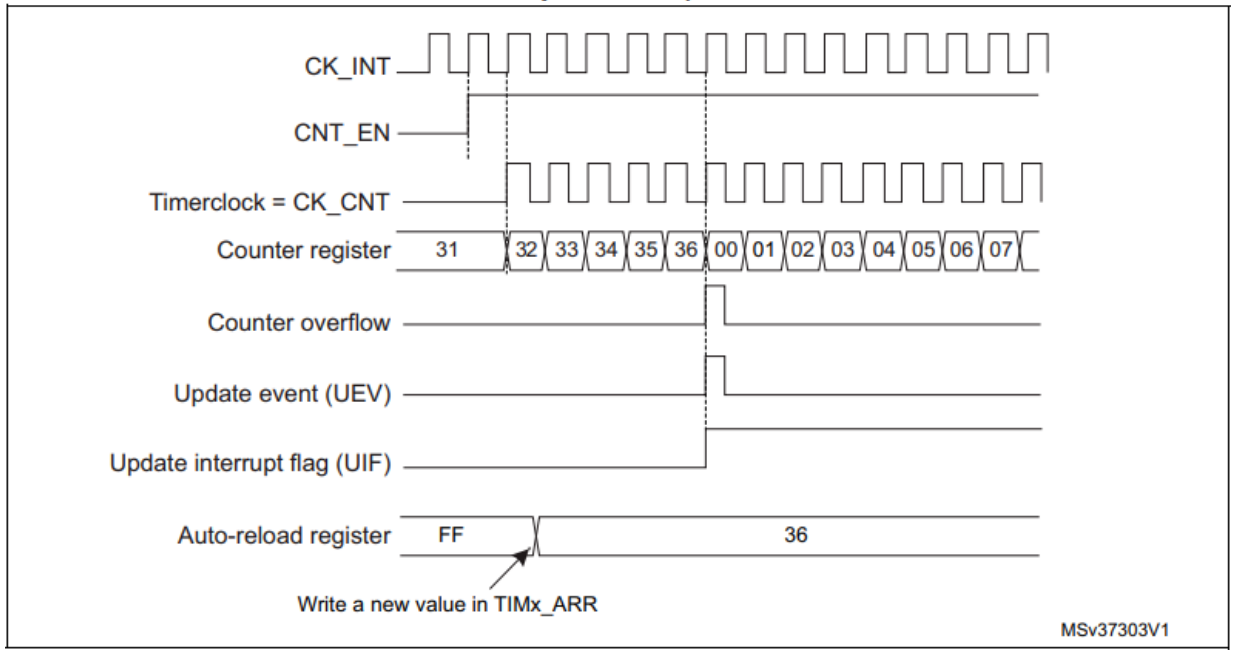
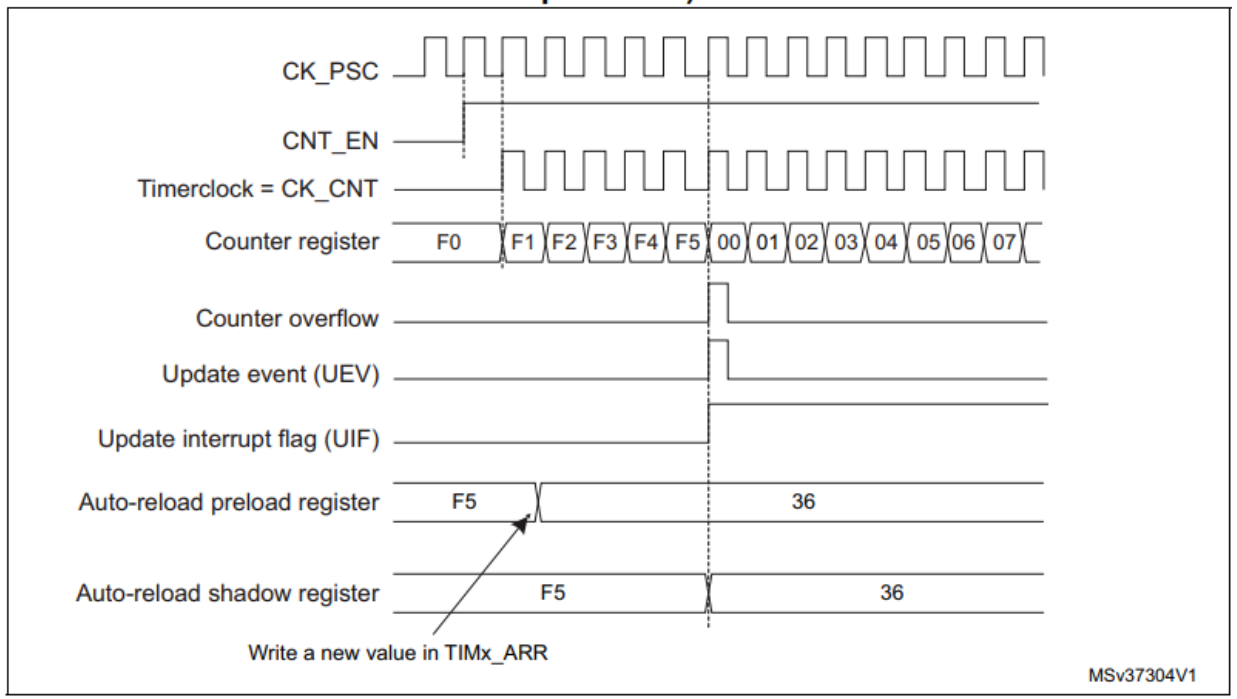


Figure 108. Counter timing diagram, Update event when ARPE=1 (TIMx_ARR preloaded)



Timer Capture/Compare

>> Input Capture

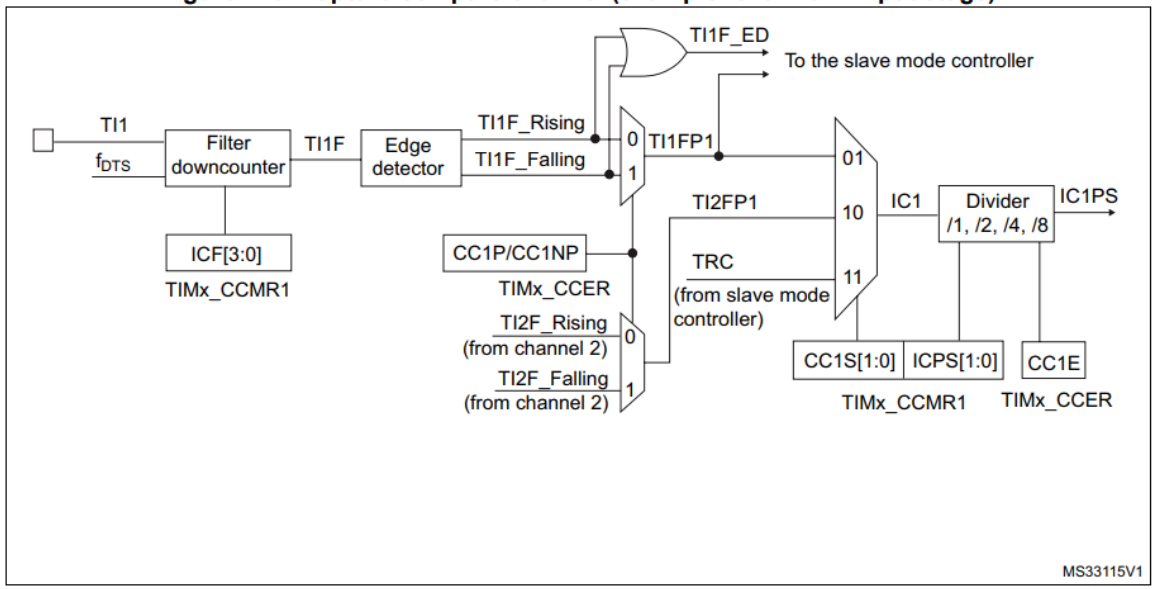
- Input Capture: 설정한 외부 입력 ICx에 대해 Rising/Falling edge를 감지하여, 해당 순간의 카운터 값을 저장

- Capture시 CCxIF가 set되고, 만약 이미 set되어 있을 경우 over-capture flag인 CCxOF가 set

CC1 channel configured as input:
This bit selects whether IC1 or IC1 is used for trigger or capture operations.
0: non-inverted: capture is done on a rising edge of IC1. When used as external trigger, IC1 is non-inverted.
1: inverted: capture is done on a falling edge of IC1. When used as external trigger, IC1 is inverted.

If channel CC1 is configured as input:
This bit is set by hardware on a capture. It is cleared by software or by reading the TIMx_CCR1 register.
0: No input capture occurred.
1: The counter value has been captured in TIMx_CCR1 register (An edge has been detected on IC1 which matches the selected polarity).

Figure 125. Capture/compare channel (example: channel 1 input stage)



Timer Capture/Compare

>> Output Compare

- Output Compare: 카운터의 값이 지정된 비교값 CCRx과 일치하는 순간 인터럽트 발생 혹은 출력을 발생

- OCxREF(출력)는 Frozen(변화없음), HIGH, LOW, Toggle동작 가능(OCxM 레지스터)

- CCxIF 인터럽트 발생

Bits 6:4 **OC1M**: Output compare 1 mode

These bits define the behavior of the output reference signal OC1REF from which OC1 and OC1N are derived. OC1REF is active high whereas OC1 and OC1N active level depends on CC1P and CC1NP bits.

000: Frozen - The comparison between the output compare register TIMx_CCR1 and the counter TIMx_CNT has no effect on the outputs.(this mode is used to generate a timing base).

001: Set channel 1 to active level on match. OC1REF signal is forced high when the counter TIMx_CNT matches the capture/compare register 1 (TIMx_CCR1).

010: Set channel 1 to inactive level on match. OC1REF signal is forced low when the counter TIMx_CNT matches the capture/compare register 1 (TIMx_CCR1).

011: Toggle - OC1REF toggles when TIMx_CNT=TIMx_CCR1.

100: Force inactive level - OC1REF is forced low.

101: Force active level - OC1REF is forced high.

110: PWM mode 1 - In upcounting, channel 1 is active as long as TIMx_CNT<TIMx_CCR1 else inactive. In downcounting, channel 1 is inactive (OC1REF='0) as long as TIMx_CNT>TIMx_CCR1 else active (OC1REF=1).

111: PWM mode 2 - In upcounting, channel 1 is inactive as long as TIMx_CNT<TIMx_CCR1 else active. In downcounting, channel 1 is active as long as TIMx_CNT>TIMx_CCR1 else inactive.

Note: In PWM mode 1 or 2, the OCREF level changes only when the result of the comparison changes or when the output compare mode switches from "frozen" mode to "PWM" mode.

Bit 1 **CC1IF**: Capture/compare 1 interrupt flag

If channel CC1 is configured as output:

This flag is set by hardware when the counter matches the compare value, with some exception in center-aligned mode (refer to the CMS bits in the TIMx_CR1 register description). It is cleared by software.

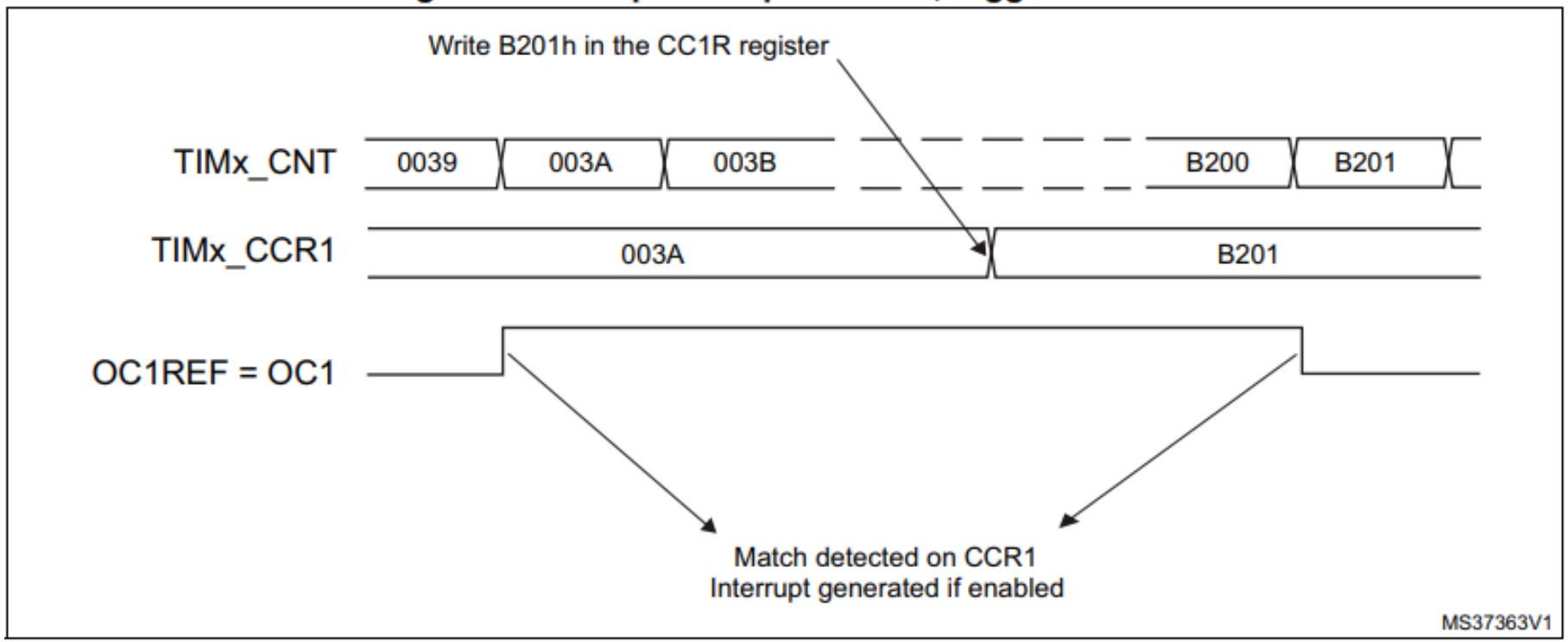
0: No match.

1: The content of the counter TIMx_CNT has matched the content of the TIMx_CCR1 register.

Timer Capture/Compare

>> Output Compare

Figure 129. Output compare mode, toggle on OC1



Timer PWM

>> Pulse Width Modulation

- PWM: Pulse Width Modulation: PWM은 ARR값을 통해 주파수를 결정, CCRx를 통해 듀티 사이클을 결정하여 최종적인 신호를 생성
- OCx별 PWM 출력 설정 가능(OCxM 110, 111 확인)

Figure 130. Edge-aligned PWM waveforms (ARR=8)

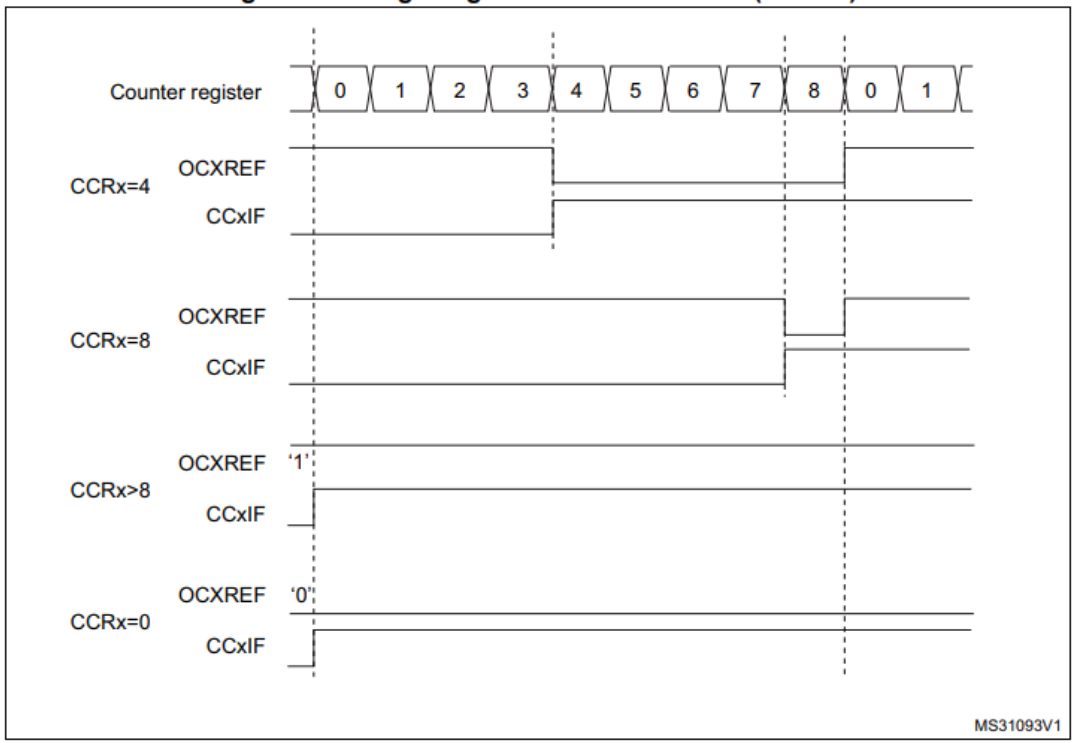
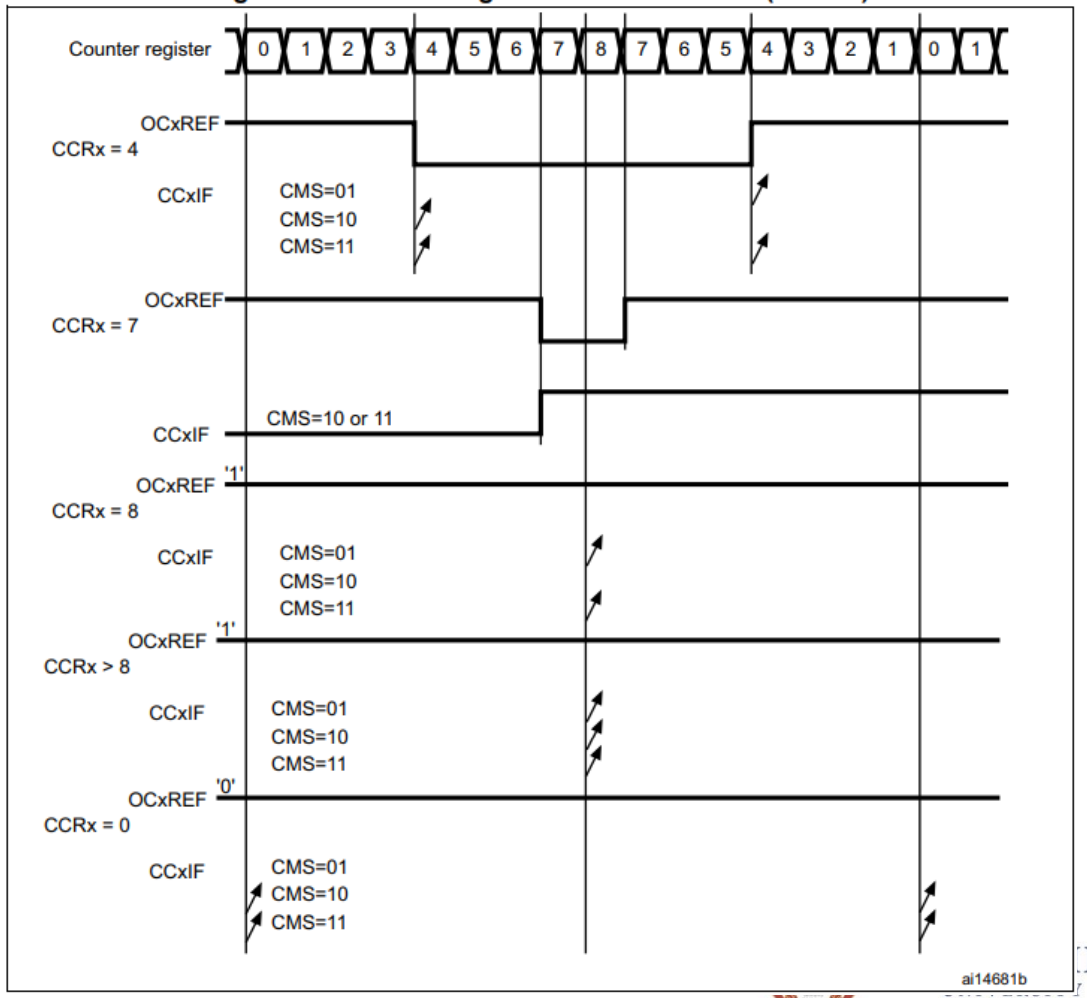


Figure 131. Center-aligned PWM waveforms (ARR=8)



Timer Base

>> Timer

- TIM1은 APB2 클럭을 사용
- TIM2, 3, 4 APB1 클럭을 사용
- Timer의 타이밍을 조절하기 위해서는 사용되는 클럭 확인이 필수

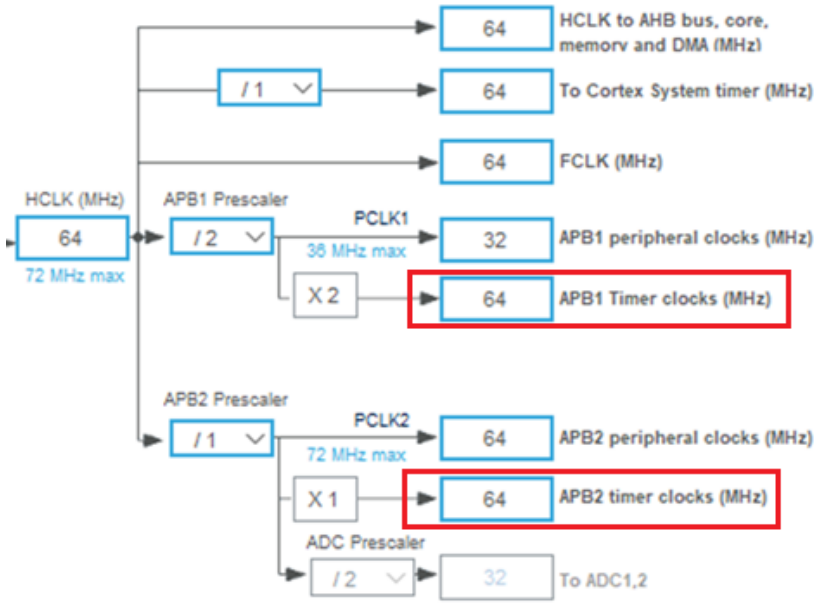
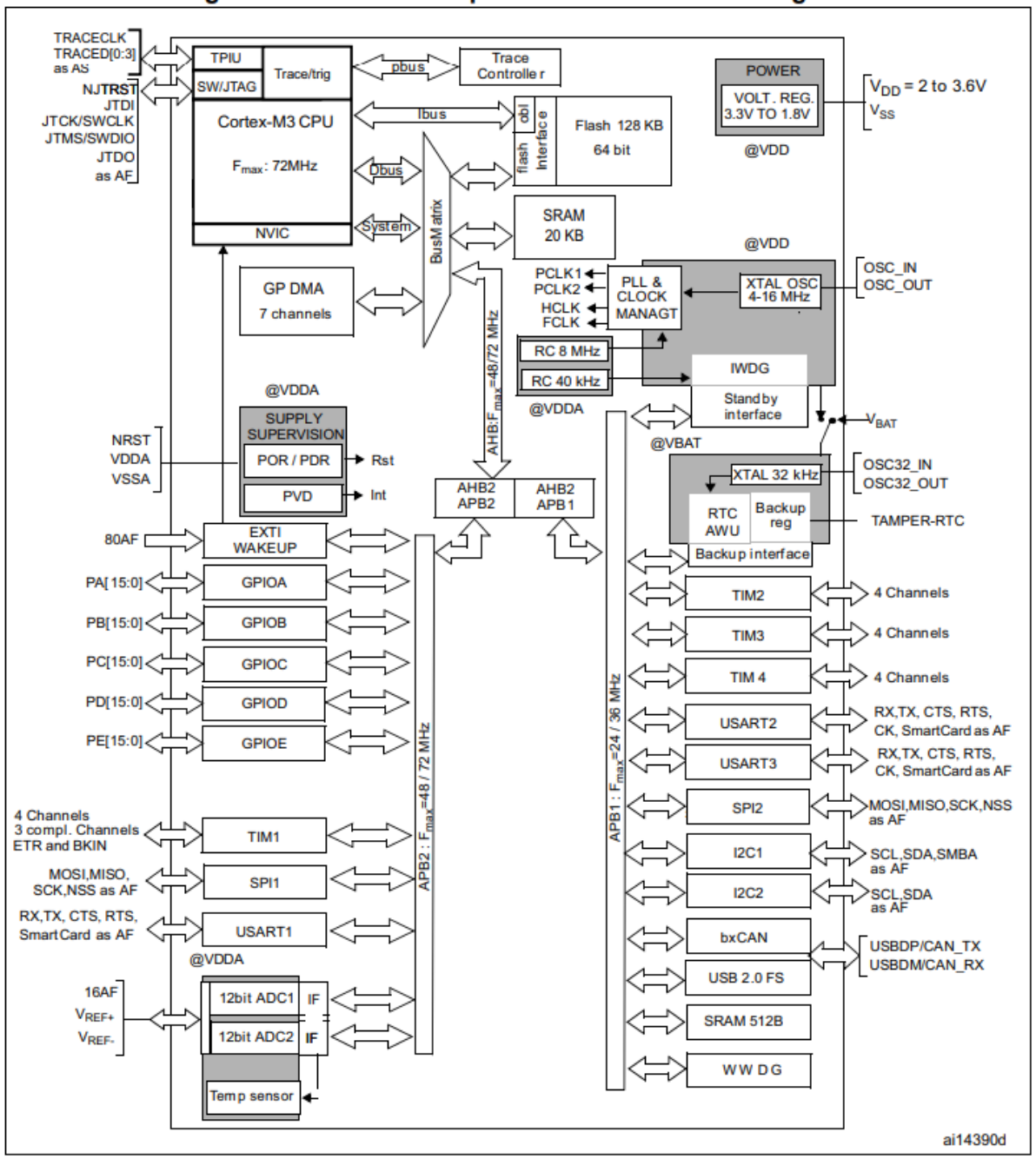


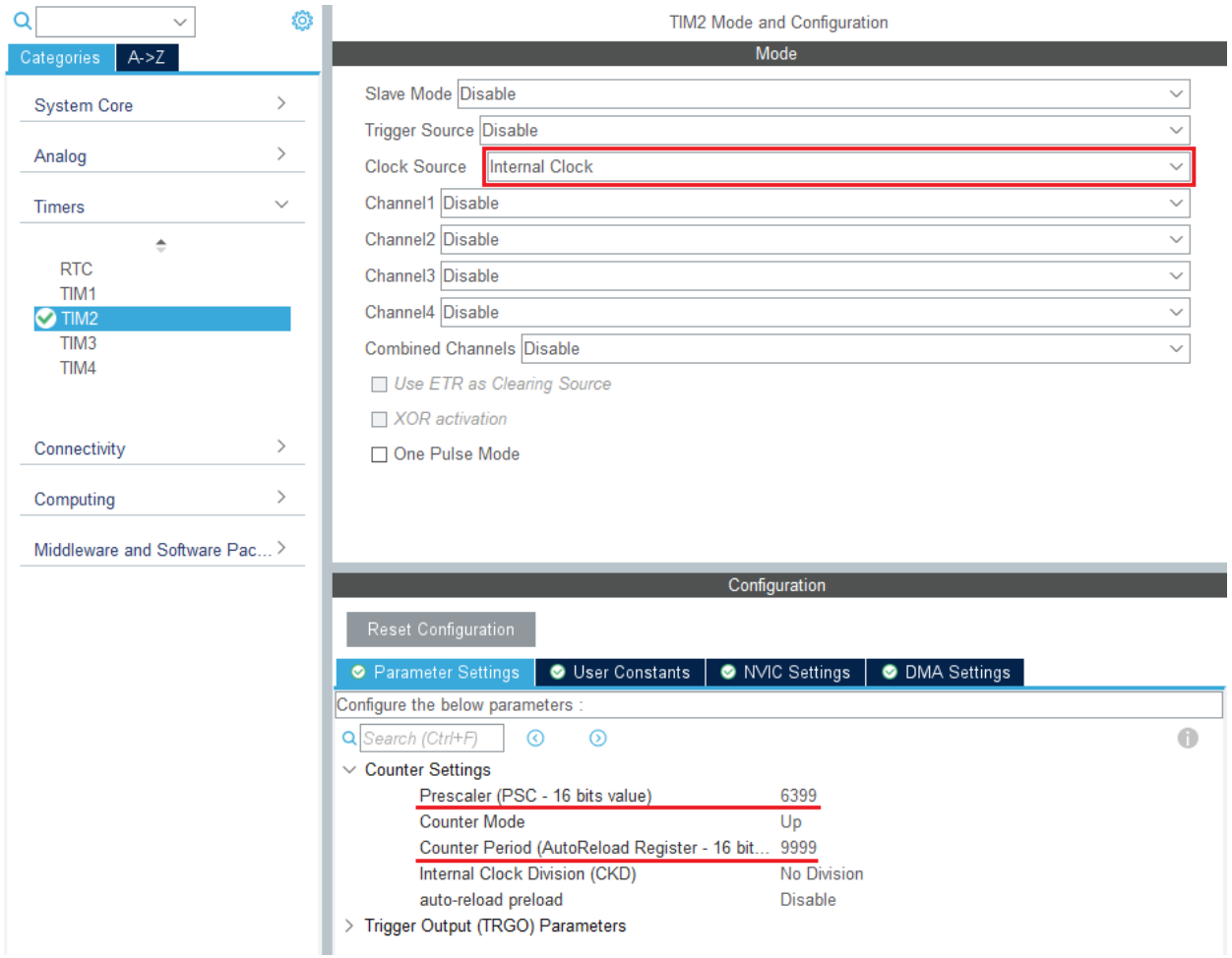
Figure 1. STM32F103xx performance line block diagram

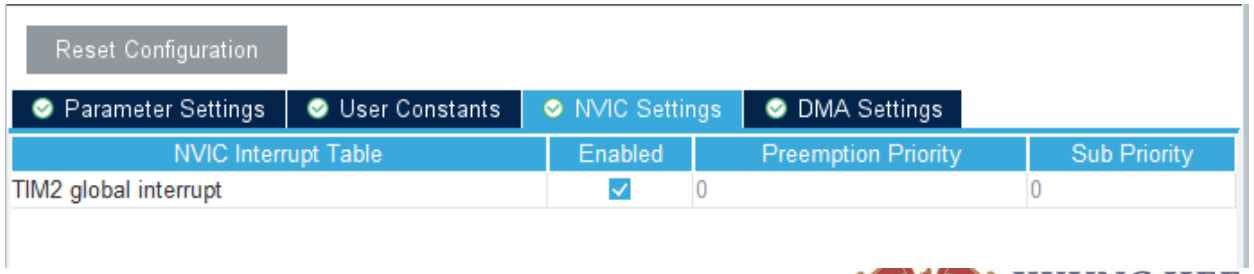


Timer Base

>> Timer

- Timer 실습 – Blink
- 1초에 마다 LED 상태 토글
- APB1 CLK = 64MHz
- CK_CNT = 64MHz / (PSC + 1) = 10KHz
- ARR = 9999
- UIF 주기 = 10KHz / (ARR + 1) = 1Hz
  - 10KHz 클럭으로 0~ARR값까지 후 카운터 초기화





Timer Base

>> Timer

TIM_Blink.ioc | main.c | stm32f1xx_it.c | stm32f1xx_hal_tim.c

```c
void HAL_TIM_IRQHandler(TIM_HandleTypeDef *htim)
{
  uint32_t itsource = htim->Instance->DIER;
  uint32_t itflag   = htim->Instance->SR;

  /* Capture compare 1 event */
  if ((itflag & (TIM_FLAG_CC1)) == (TIM_FLAG_CC1))
  {
    if ((itsource & (TIM_IT_CC1)) == (TIM_IT_CC1))
    {
      {
        __HAL_TIM_CLEAR_FLAG(htim, TIM_FLAG_CC1);
        htim->Channel = HAL_TIM_ACTIVE_CHANNEL_1;

        /* Input capture event */
        if ((htim->Instance->CCMR1 & TIM_CCMR1_CC1S) != 0x00U)
        {
#if (USE_HAL_TIM_REGISTER_CALLBACKS == 1)
          htim->IC_CaptureCallback(htim);
#else
          HAL_TIM_IC_CaptureCallback(htim);
#endif /* USE_HAL_TIM_REGISTER_CALLBACKS */
        }
        /* Output compare event */
        else
        {
#if (USE_HAL_TIM_REGISTER_CALLBACKS == 1)
          htim->OC_DelayElapsedCallback(htim);
          htim->PWM_PulseFinishedCallback(htim);
#else
          HAL_TIM_OC_DelayElapsedCallback(htim);
          HAL_TIM_PWM_PulseFinishedCallback(htim);
#endif /* USE_HAL_TIM_REGISTER_CALLBACKS */
        }
        htim->Channel = HAL_TIM_ACTIVE_CHANNEL_CLEARED;
      }
    }
```

```c
  /* TIM Update event */
  if ((itflag & (TIM_FLAG_UPDATE)) == (TIM_FLAG_UPDATE))
  {
    if ((itsource & (TIM_IT_UPDATE)) == (TIM_IT_UPDATE))
    {
      __HAL_TIM_CLEAR_FLAG(htim, TIM_FLAG_UPDATE);
#if (USE_HAL_TIM_REGISTER_CALLBACKS == 1)
      htim->PeriodElapsedCallback(htim);
#else
      HAL_TIM_PeriodElapsedCallback(htim);
#endif /* USE_HAL_TIM_REGISTER_CALLBACKS */
```

```c
  uint32_t itsource = htim->Instance->DIER;
  uint32_t itflag   = htim->Instance->SR;

```

```c
#define TIM_IT_UPDATE                    TIM_DIER_UIE
```

Bit 0 **UIF**: Update interrupt flag

- This bit is set by hardware on an update event. It is cleared by software.
  0: No update occurred.
  1: Update interrupt pending. This bit is set by hardware when the registers are updated:
- At overflow or underflow and if the UDIS=0 in the TIMx_CR1 register.
- When CNT is reinitialized by software using the UG bit in TIMx_EGR register, if URS=0 and UDIS=0 in the TIMx_CR1 register.
- When CNT is reinitialized by a trigger event (refer to the synchro control register description), if URS=0 and UDIS=0 in the TIMx_CR1 register.

Timer Base

>> Timer

- Weak로 선언된 PeriodElapsedCallback 함수 동작 정의
- 카운터가 ARR에 도달할 때 마다 Toggle 동작
- HAL_TIM_Base_Start IT를 통해 TIM 인터럽트 동작 시작
- 보드의 내장 LED를 통해 동작 확인

```
  /* USER CODE BEGIN 2 */

  HAL_TIM_Base_Start_IT(&htim2);

  /* USER CODE END 2 */
```

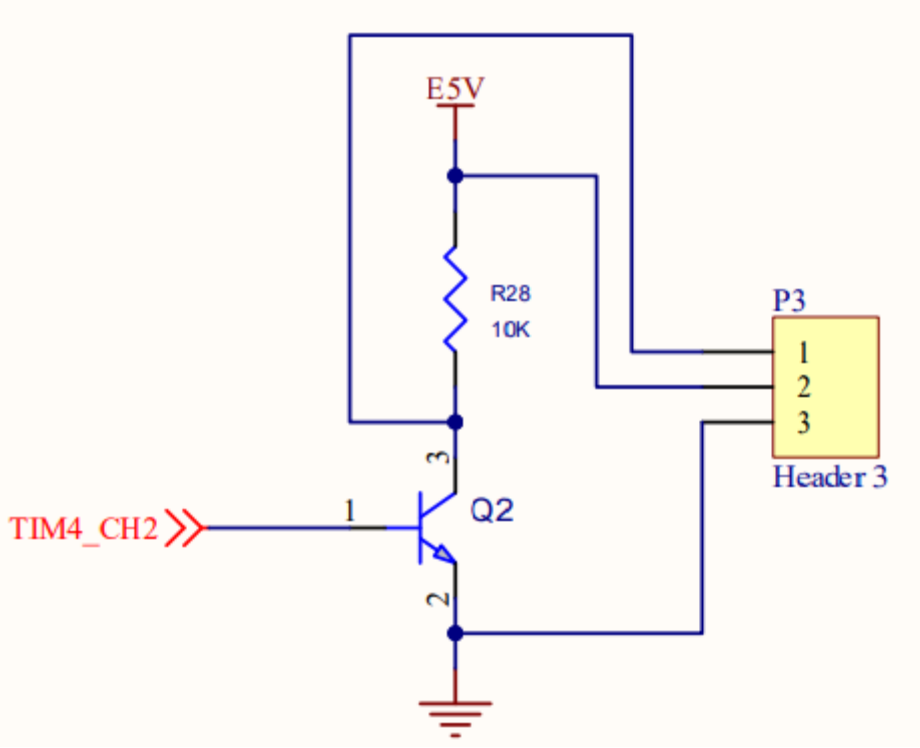
```

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2)
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
}
/* USER CODE END 4 */

```

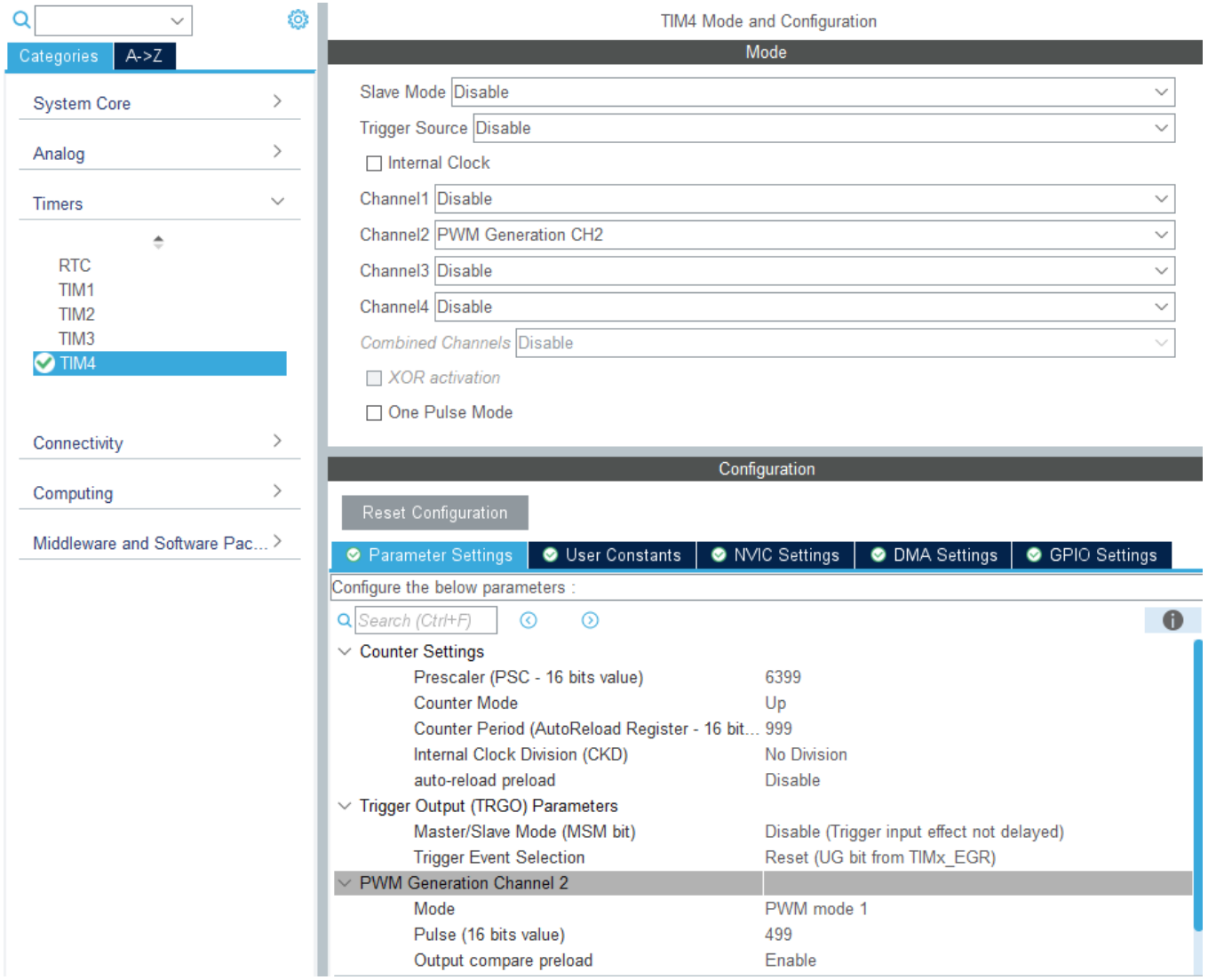
KYUNG HEE UNIVERSITY

Timer PWM

>> Timer

- **PWM 실습 – PWM 파형 확인**

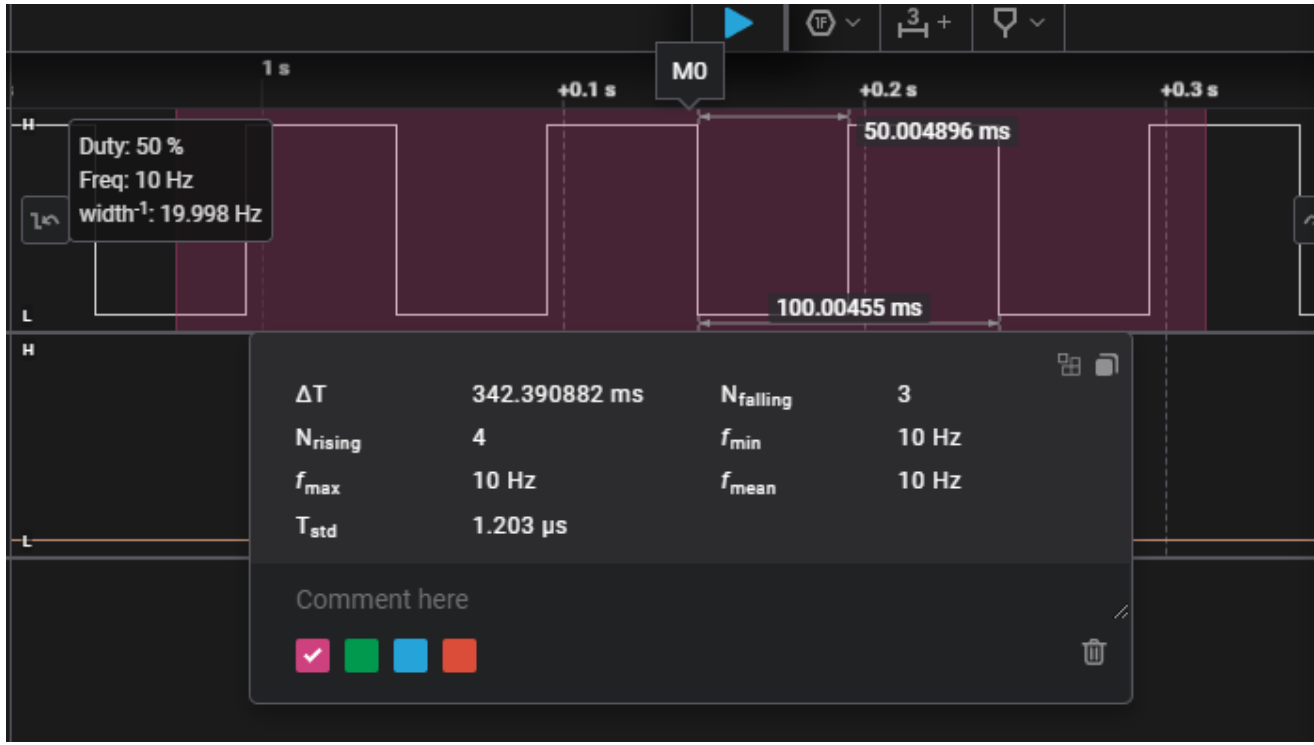




Timer PWM

>> Timer

- APB1 CLK = 64MHz
- CK_CLK = 64MHz / (6399 + 1) = 10KHz
- ARR = 999, 최종 클럭10Hz
- PWM Pulse 500, Pulse / (ARR + 1) = 50%
- 50% Duty cycle PWM signal -> 로직분석기 활용



Counter Settings	
Prescaler (PSC - 16 bits value)	6399
Counter Mode	Up
Counter Period (AutoReload Register - 16 bit...	999
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable
Trigger Output (TRGO) Parameters	
PWM Generation Channel 2	
Mode	PWM mode 1
Pulse (16 bits value)	500
Output compare preload	Enable
Fast Mode	Disable
CH Polarity	High

110: PWM mode 1 - In upcounting, channel 1 is active as long as TIMx_CNT<TIMx_CCR1 else inactive. In downcounting, channel 1 is inactive (OC1REF='0) as long as TIMx_CNT>TIMx_CCR1 else active (OC1REF=1).
111: PWM mode 2 - In upcounting, channel 1 is inactive as long as TIMx_CNT<TIMx_CCR1 else active. In downcounting, channel 1 is active as long as TIMx_CNT>TIMx_CCR1 else inactive.

```
/* USER CODE BEGIN 2 */

if (HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_2) != HAL_OK)
    Error_Handler();

/* USER CODE END 2 */
```

HEE UNIVERSITY

Timer PWM

>> Timer

- 다양한 Pulse값으로 수정 후 실험
- PWM mode 2 비교 실습
- 기본 주파수 변경 실습

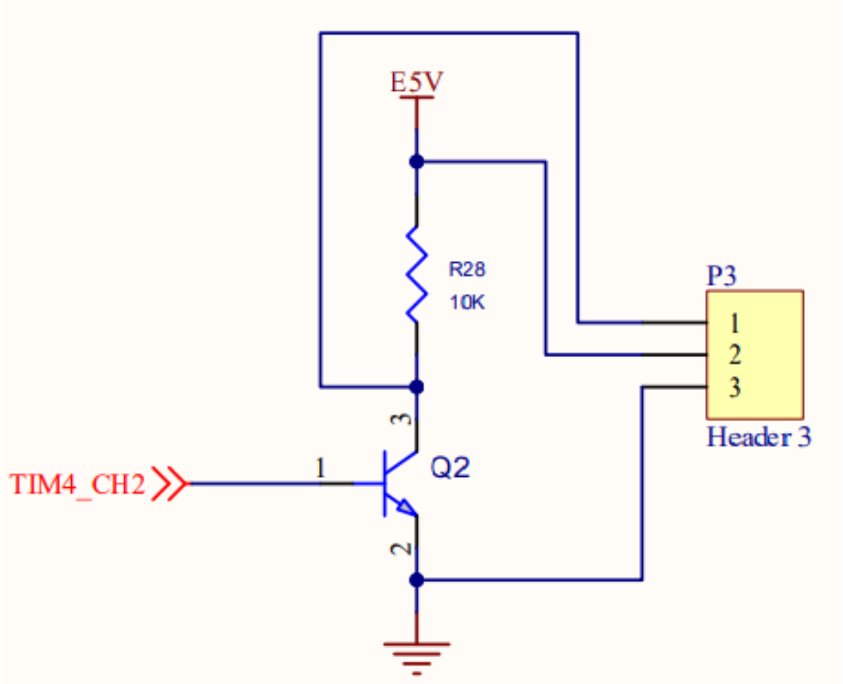
```
sConfigOC.OCMode = TIM_OCMODE_PWM1;
sConfigOC.Pulse = 123;
sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
if (HAL_TIM_PWM_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
```

Timer PWM

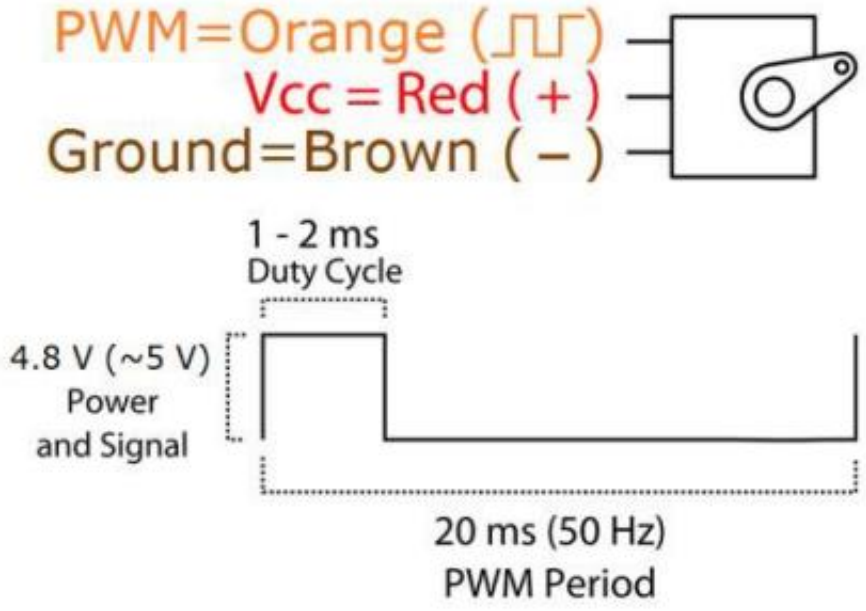
>> Timer

- 확장 보드 실습 – PWM을 통한 Servo motor 제어

- 50Hz, 5%~10% Duty Cycle을 통해 각도 제어



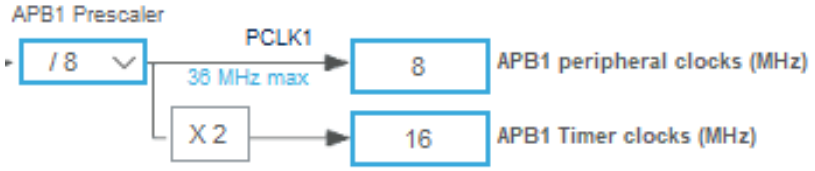
Position "0" (1.5 ms pulse) is middle, "90" (~2ms pulse) is middle,
is all the way to the right, "-90" (~1ms pulse) is all the way to the left.



Timer PWM

>> Timer

- APB1 CLK = 16MHz 설정
- 1MHz = 16MHz / (PSC + 1), PSC = 15
- 1MHz / 50Hz = 20000, ARR = 19999
- Duty cycle 5%: 20000 * 5% = 1000
- Duty cycle 10%: 20000 * 10% = 2000
- Pulse 값을 1000~2000 사이로 조절하여 -90˚ ~ 90˚ 제어
- 확장 보드의 회로도(Q2)에 따라 CH Polarity – Low 설정



Counter Settings	
Prescaler (PSC - 16 bits value)	15
Counter Mode	Up
Counter Period (AutoReload R...	19999
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable
Trigger Output (TRGO) Parameters	
PWM Generation Channel 2	
Mode	PWM mode 1
Pulse (16 bits value)	1500
Output compare preload	Enable
Fast Mode	Disable
CH Polarity	Low

자료는 SG90 모터 기준. 일부 다른 모터(ex. SG92R)의 경우 0.5 ~ 2.5ms duty cycle을 사용해야함. 사용 부품 데이터 시트 확인 필수

Timer PWM

>> Timer

- __HAL_TIM_GET_COMPARE를 사용해서 CCRx 값을 변경 가능

- 제어하고자 하는 각도에 따라 적절한 pulse값으로 수정하여 서보 모터 제어

- 다양한 각도 출력 및 확인, -90° ~ 90° 좌우 왕복 실습

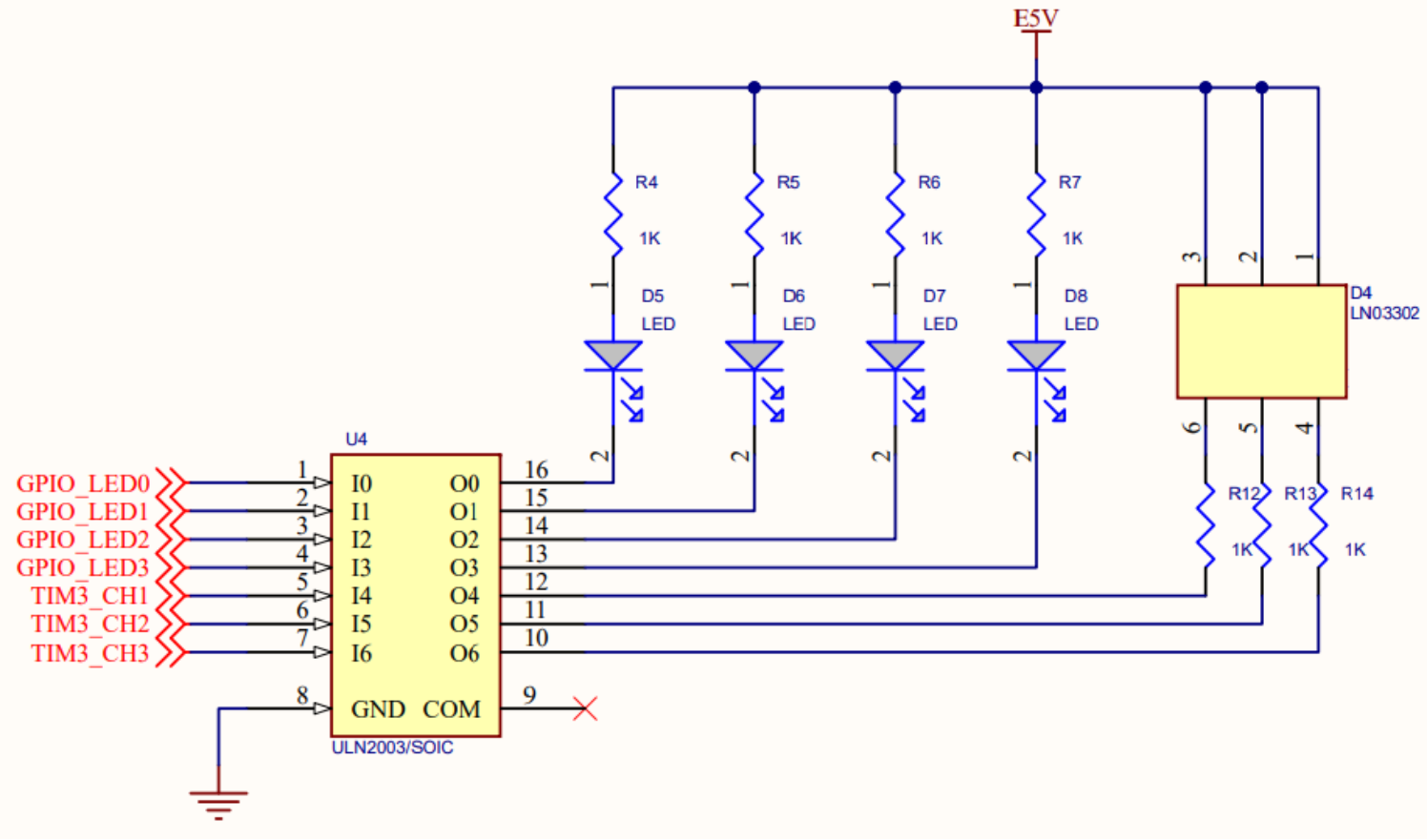
```
pulse = 1600;
__HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, pulse);
```

```
#define __HAL_TIM_SET_COMPARE(__HANDLE__, __CHANNEL__, __COMPARE__) \
  (((__CHANNEL__) == TIM_CHANNEL_1) ? ((__HANDLE__)->Instance->CCR1 = (__COMPARE__)) :\
   ((__CHANNEL__) == TIM_CHANNEL_2) ? ((__HANDLE__)->Instance->CCR2 = (__COMPARE__)) :\
   ((__CHANNEL__) == TIM_CHANNEL_3) ? ((__HANDLE__)->Instance->CCR3 = (__COMPARE__)) :\
   ((__HANDLE__)->Instance->CCR4 = (__COMPARE__)))
```

Timer PWM

>> Timer

- 확장 보드 실습 – RGB LED 밝기, PWM 제어



Timer PWM

>> Timer

Categories | A->Z

- System Core
- Analog
- Timers
  - RTC
  - TIM1
  - TIM2
  - TIM3
  - TIM4
- Connectivity
- Computing
- Middleware and Software Packs

TIM3 Mode and Configuration

Mode

Channel1	PWM Generation CH1
Channel2	PWM Generation CH2
Channel3	PWM Generation CH3
Channel4	Disable
Combined Channels	Disable

☐ Use ETR as Clearing Source

☐ XOR activation

☐ One Pulse Mode

Configuration

Reset Configuration

NVIC Settings | DMA Settings | GPIO Settings

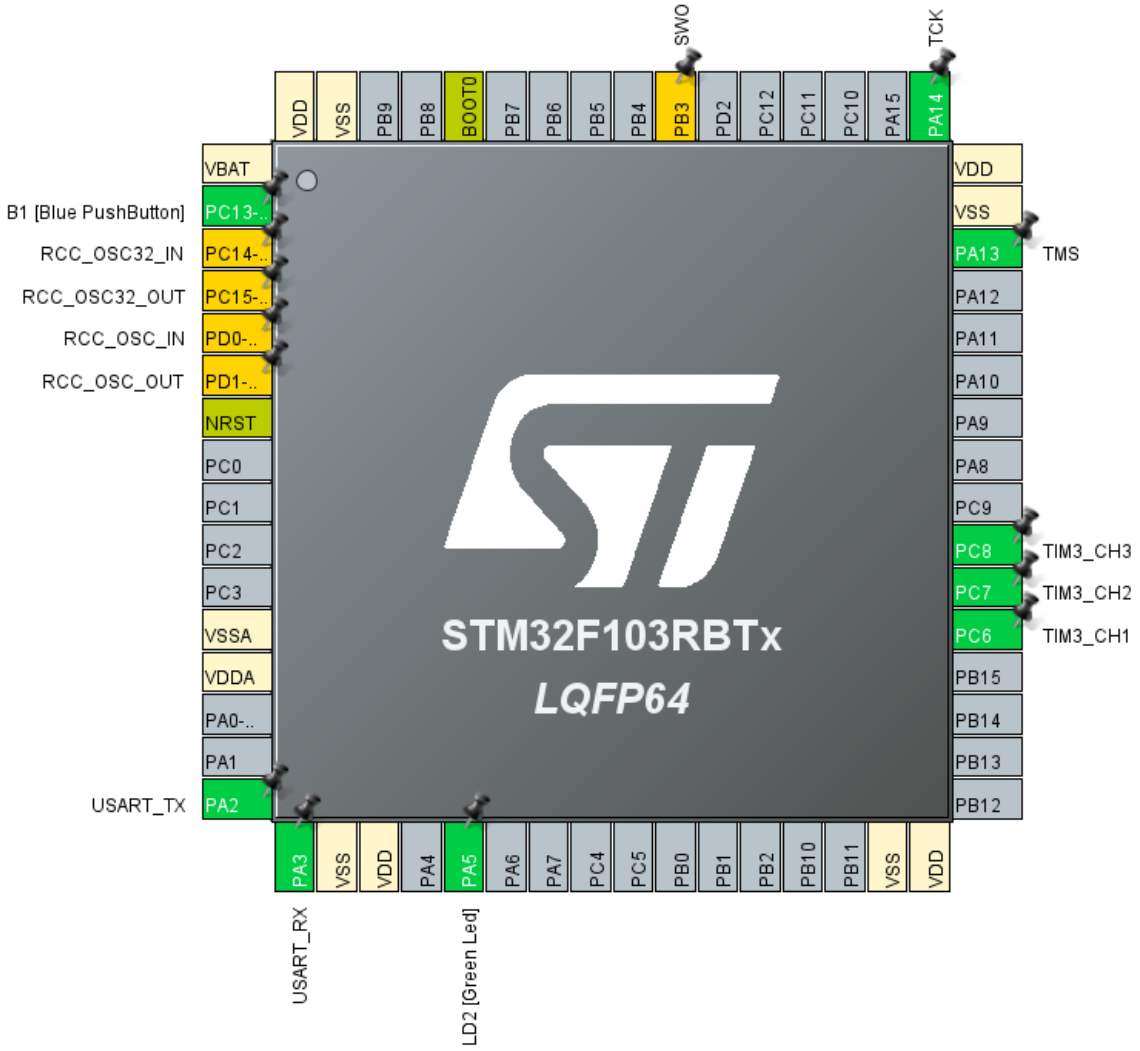
Parameter Settings | User Constants

Configure the below parameters :

Search (Ctrl+F)

Counter Settings	
Prescaler (PSC - 16 bits ...	63
Counter Mode	Up
Counter Period (AutoRelo...	999
Internal Clock Division (C...	No Division
auto-reload preload	Disable
Trigger Output (TRGO) Parameters	
PWM Generation Channel 1	
Mode	PWM mode 1
Pulse (16 bits value)	0
Output compare preload	Enable
Fast Mode	Disable
CH Polarity	High
PWM Generation Channel 2	
Mode	PWM mode 1
Pulse (16 bits value)	0

Pinout view | System view



Timer PWM

>> Timer

- CubeMX TIM3 PWM CH 출력 지정 확인

- 시간에 따라 R, G, B 밝기 순회

```
  /* USER CODE BEGIN 2 */

  HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
  HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);
  HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_3);

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
      for (uint16_t i = 0; i < 1000; i++) {
          __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 999 - i);
          __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, i);
          HAL_Delay(5);
      }

      for (uint16_t i = 0; i < 1000; i++) {
          __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 999 - i);
          __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, i);
          HAL_Delay(5);
      }

      for (uint16_t i = 0; i < 1000; i++) {
          __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, 999 - i);
          __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, i);
          HAL_Delay(5);
      }
  }
  /* USER CODE END 3 */
}
```

Timer PWM

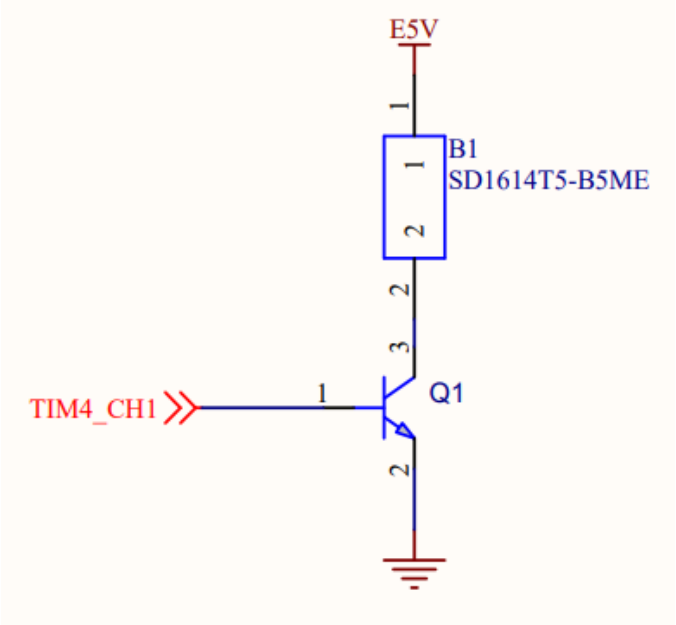
>> Timer

- 확장 보드 실습 – Buzzer

- APB1 CLK = 64MHz

- PSC = 63, CNT_CLK 1MHz

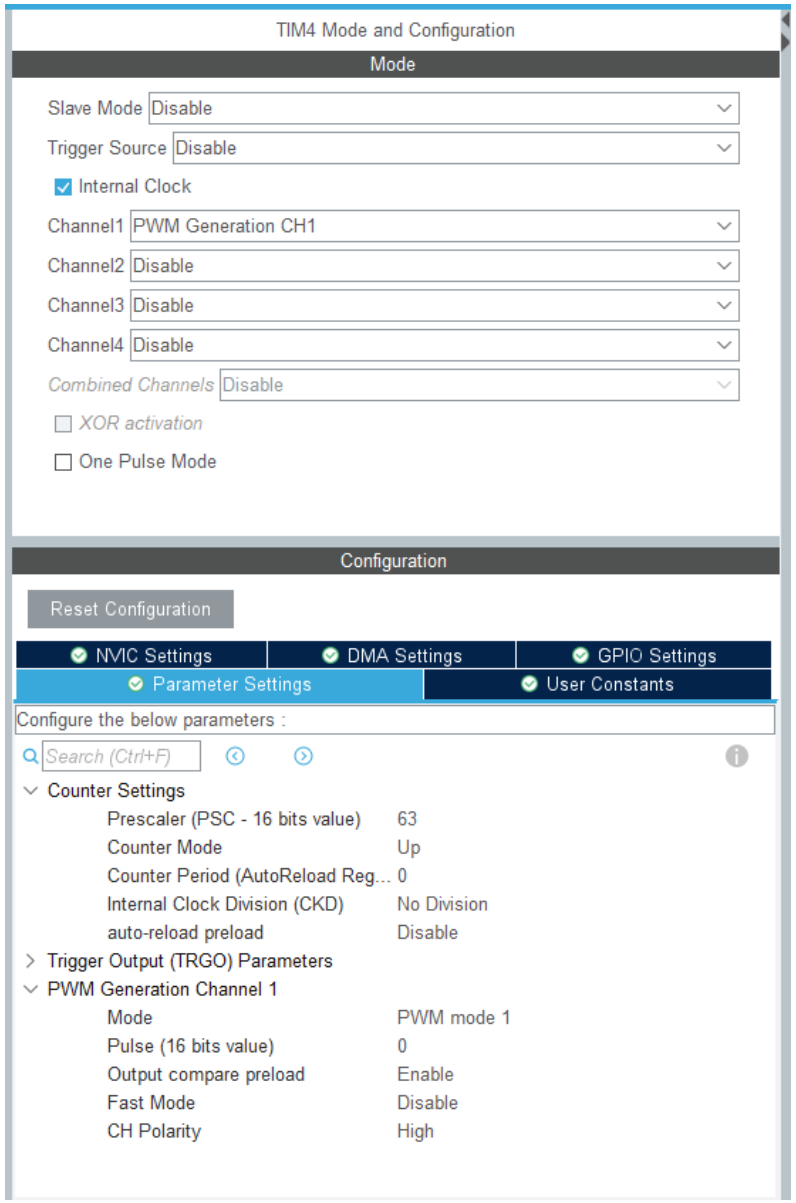
- ARR값을 조정하여 음계 출력
  - $\frac{1\text{MHz}}{\text{tone}} = \text{ARR} + 1$
  - PWM 50%
  - Pulse = (ARR + 1) / 2



음 이름	계이름	주파수 (Hz)
C4	도	261.63
D4	레	293.66
E4	미	329.63
F4	파	349.23
G4	솔	392
A4	라	440
B4	시	493.88

Timer PWM

>> Timer




```c
#define CNT_CLK 1000000

/* USER CODE END PD */

/* Private macro -------------------------------------------
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------
TIM_HandleTypeDef htim4;

UART_HandleTypeDef huart2;

/* USER CODE BEGIN PV */

enum notes {
    C4 = 26163,
    D4 = 29366,
    E4 = 32963,
    F4 = 34923,
    G4 = 39200,
    A4 = 44000,
    B4 = 49388,
    C5 = 52325
};

int melody[] = {G4, G4, A4, A4, G4, G4, E4, G4, G4, E4, E4, D4,
                G4, G4, A4, A4, G4, G4, E4, G4, E4, D4, E4, C4};
```

```c
  /* USER CODE BEGIN 2 */

  int freq;

  for (uint8_t i = 0; i < sizeof(melody) / sizeof(melody[0]); i++) {
      freq = (int)(CNT_CLK / melody[i] * 100);
      __HAL_TIM_SET_AUTORELOAD(&htim4, freq);
      __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, (int)(freq / 2));
      HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_1);
      HAL_Delay(500);

      HAL_TIM_PWM_Stop(&htim4, TIM_CHANNEL_1);
      HAL_Delay(10);

  }

  /* USER CODE END 2 */
```

JNG HEE
IVERSITY

Timer PWM

>> Timer

- **Mario – overworld**
- **음계간 딜레이 적절하게 조절**

```
enum notes {
C0 = 1635, CS0 = 1732, D0 = 1835, DS0 = 1945, E0 = 2060, F0 = 2183, FS0 = 2310, G0 = 2450, GS0 = 2596, A0 = 2750, AS0 = 2914, B0 =
3087,
C1 = 3270, CS1 = 3465, D1 = 3671, DS1 = 3889, E1 = 4120, F1 = 4365, FS1 = 4620, G1 = 4900, GS1 = 5196, A1 = 5500, AS1 = 5827, B1 =
6174,
C2 = 6541, CS2 = 6930, D2 = 7342, DS2 = 7778, E2 = 8241, F2 = 8731, FS2 = 9240, G2 = 9800, GS2 = 10392, A2 = 11000, AS2 = 11654, B2
= 12347,
C3 = 13081, CS3 = 13859, D3 = 14683, DS3 = 15556, E3 = 16481, F3 = 17461, FS3 = 18500, G3 = 19600, GS3 = 20765, A3 = 22000, AS3 =
23308, B3 = 24694,
C4 = 26163, CS4 = 27718, D4 = 29366, DS4 = 31113, E4 = 32963, F4 = 34923, FS4 = 36999, G4 = 39200, GS4 = 41530, A4 = 44000, AS4 =
46616, B4 = 49388,
C5 = 52325, CS5 = 55437, D5 = 58733, DS5 = 62225, E5 = 65926, F5 = 69846, FS5 = 73999, G5 = 78399, GS5 = 83061, A5 = 88000, AS5 =
93233, B5 = 98777,
C6 = 104650, CS6 = 110873, D6 = 117466, DS6 = 124451, E6 = 131851, F6 = 139691, FS6 = 147998, G6 = 156798, GS6 = 166122, A6 = 176000,
AS6 = 186466, B6 = 197553,
C7 = 209300, CS7 = 221746, D7 = 234932, DS7 = 248902, E7 = 263702, F7 = 279383, FS7 = 295996, G7 = 313596, GS7 = 332244, A7 = 352000,
AS7 = 372931, B7 = 395107,
C8 = 418600, CS8 = 443492, D8 = 469864, DS8 = 497803, E8 = 527404, F8 = 558765, FS8 = 591992, G8 = 627193, GS8 = 664488, A8 = 704000,
AS8 = 745862, B8 = 790214
};

int melody[] = {E5, E5, E5, C5, E5, G5, G4,
C5, G4, E4, A4, B4, AS4, A4,
G4, E5, G5, A5, F5, G5, E5,
C5, D5, B4};
```

Timer

>> Timer

- Timer 실습 과제 1
  - 3개의 버튼에 따라 -90˚, 0, 90˚ 출력 (EXIT 사용)

- Timer 실습 과제 2
  - AD 변환 완료 인터럽트를 활용
  - 가변저항을 통해 서보모터 제어

- Timer 실습 과제 3
  - 터치 센서를 통해 부저 소리 On/Off 제어(EXIT)
  - 로터리 엔코더를 사용하여 부저 제어
  - 시계방향 회전시 점차 고음, 반시계 방향 회전시 점차 저음으로 변경

>> Timer

- Timer 실습 과제 4
  - 버튼 4, RGB LED, 가변 저항 활용
  - 1, 2, 3번 버튼을 통해 R, G ,B LED를 선택(설정 모드, EXIT)
  - 선택된 색의 LED를 가변저항에 따라 밝기를 제어 및 출력
  - 다른 버튼을 통해 다른 색의 LED 설정모드로 변경
  - 4번 버튼을 누를 시 설정된 LED값을 고정(고정되어 조합된 3색 LED 출력)

Thank you

Department of Electronic Engineering,
Kyung Hee University, 1732, Deogyeong-Daero,
Giheung-gu, Yongin-si, Gyeonggi-Do, 17104,
Republic of Korea
Web: https://khu-hri.weebly.com/

Human-Robot Interaction Laboratory

KYUNG HEE UNIVERSITY